SIEM LAB · VOLUME 5 · DETECTION COMPANION

The Wazuh Detection Companion for Windows Server Attacks

The blue side of your lab, as a reference. Every technique you practise in Volumes 3–4 is listed here with the Windows Event IDs that expose it, the exact Wazuh dashboard search that finds it, and the rule to write if the stock rules miss it.

Side of the bench	**Defender / detection** — read this while logged into Wazuh
SIEM	SIEM-Ubuntu `10.0.0.40` — Wazuh 4.14.8 (manager, indexer, dashboard, filebeat)
Monitored hosts	WinServer DC `10.0.0.10` (agent `002` "Server"), Kali `10.0.0.30` (agent `001`)
Dashboard	`https://127.0.0.1:4433` — `admin` / `<MANAGER_PASSWORD>`
How to use it	Attack with Volume 4 open, keep this as the lookup table beside it. Every search below is copy-paste into the Alerts search bar.

> **Detection is the safer start.** This document contains no attack commands — only the signs to watch for. Pair it with Volume 4: it is what the dashboard looks like from the chair you should deserve to sit in.

Scope — what this reference covers

One page of truth first, then the matrix. Wazuh does not invent Windows telemetry — it ships what the Windows auditing subsystem records. The difference between a quiet lab and a deafening one is almost always **audit policy being turned on**.

The five Wazuh fields you will actually use

Field	
`agent.name`	Which machine (use <SERVER> or the agent name you chose).
`data.win.system.eventID`	The Windows Event ID (e.g. `4625`). Older indexes: `win.system.eventID`.
`data.win.system.severity`	The source's own severity hint, if present.
`rule.level`	Wazuh's 0–15 assessment. 8+ = investigate, 12+ = respond.
`rule.groups`	Tags like `windows`, `sysmon`, `authentication_failures`.

To confirm which field names your install uses, click any `4625` alert and inspect its JSON — it takes ten seconds and removes all guesswork.

Contents

1 Making sure Windows and Wazuh are actually talking (audit policy first)

2 The detection matrix — technique → Event IDs → search

3 Timeline reads: four signature searches for Volume 4's phases

4 Writing local rules for the gaps (the 4662 problem, 4625 frequency)

5 Tuning: from "everything" to "everything interesting"

6 Keeping the pipeline alive (and the filebeat fix that matters)

7 Cheat sheet of searches

A Coverage table for your report

1 First: make sure Windows will tell you anything

Wazuh's agent forwards events Windows already logged. If audit policy is off, attacks slip past silently no matter how good your SIEM is. Run this on the DC (console or shell) once:

```
auditpol /get /category:* | findstr /i "Logon Advanced Audit"
```

Audit category to confirm	Key subcategories
Logon/Logoff	Logon, Logoff, Account Lockout — feeds `4624/4625/4740`
Account Logon	Kerberos Service Ticket, Kerberos Authentication Service — feeds `4768/4769`
Detailed Tracking	Process Creation (with command line) — feeds `4688` and `4104`
DS Access	Directory Service Access — feeds the DCSync `4662`
Account Management	User/Group Management — feeds `4720/4732`

1 **Turn on process command-line auditing (huge for any red-team lab):**

```
reg add "HKLM\Software\Microsoft\Windows\CurrentVersion\Policies\System\Audit" ^
  /v ProcessCreationIncludeCmdLine_Enabled /t REG_QWORD /d 1 /f
```

2 **Turn on the other categories if they read "No Auditing":**

```
auditpol /set /subcategory:"Logon" /success:enable /failure:enable
auditpol /set /subcategory:"Logoff" /success:enable /failure:enable
auditpol /set /subcategory:"Account Lockout" /success:enable /failure:enable
auditpol /set /subcategory:"Kerberos Service Ticket Operations" /success:enable
/failure:enable
auditpol /set /subcategory:"Kerberos Authentication Service" /success:enable /failure:enable
auditpol /set /subcategory:"Process Creation" /success:enable /failure:enable
auditpol /set /subcategory:"Directory Service Access" /success:enable /failure:enable
auditpol /set /subcategory:"User Account Management" /success:enable /failure:enable
auditpol /set /subcategory:"Security Group Management" /success:enable /failure:enable
```

3 **On the DC, also flag the object that DCSync touches (so `4662` fires):**

```
Set-ADObject -Identity "CN=Replicator,CN=Users,DC=corp,DC=local" ^
  -Audio:Apply- 2>$null
auditpol /set /subcategory:"Directory Service Access" /success:enable /failure:enable
```

Full replication-GUID auditing is a small project (Volume 5 does not batch it for you). The reliable lab shortcut: pair Sara's guidance on "Directory Service Access" with the search in Part 2 — seeing *some* DS-access events is a good start.

The settings that make a lab honest

No audit policy = no events = no fun. Before every campaign, confirm the DC logs `4624/4625/4688`. It takes two minutes and saves an hour of "why is Wazuh quiet?".

2 The detection matrix

Technique on the left, the Event IDs that expose it in the middle, the copy-paste search on the right. Change `<SERVER>` to your agent name.

Technique (Volume 4 phase)	Event IDs	Wazuh dashboard search
Scanning footprint (2)	5200-series kittens / noisy SMB	`agent.name:<SERVER> AND data.win.system.eventID:4625`
Username enumeration via Kerberos (3)	4768, 4771	`agent.name:<SERVER> AND data.win.system.eventID:(4768 OR 4771)`
Password spray / brute force (4.2)	4625, 4740	`agent.name:<SERVER> AND data.win.system.eventID:4625`
Lockout (went too fast) (4.2)	4740	`agent.name:<SERVER> AND data.win.system.eventID:4740`
AS-REP roasting (4.3)	4768 (odd pre-auth)	`agent.name:<SERVER> AND data.win.system.eventID:4768`
Kerberoasting (4.4)	4769 (RC4 if present)	`agent.name:<SERVER> AND data.win.system.eventID:4769`
SMB share enumeration (3.1)	— (or 4625)	`agent.name:<SERVER> AND data.win.system.eventID:4625`
Successful network logon (shell) (5)	4624	`agent.name:<SERVER> AND data.win.system.eventID:4624`
Remote command execution (5.1)	4688	`agent.name:<SERVER> AND data.win.system.eventID:4688`
PowerShell abuse (5.2)	4104, 4103	`agent.name:<SERVER> AND data.win.system.eventID:4104`
DCSync (replication read) (6.1)	4662 (replication GUIDs)	`agent.name:<SERVER> AND data.win.system.eventID:4662`
Pass-the-Hash (6.2)	4624 (logon type 3, odd session)	`agent.name:<SERVER> AND data.win.system.eventID:4624`
Account creation / group add (impact) (6.3)	4720, 4732, 4726	`agent.name:<SERVER> AND data.win.system.eventID:(4720 OR 4732 OR 4726)`
Privileged logon (happy path)	4672	`agent.name:<SERVER> AND data.win.system.eventID:4672`

> **The DCSync detail**
>
> `4662` matters only when the object GUID matches the replication pair `1131f6aa-…` / `1131f6ad-…`. On an untuned install you may see every DS access — filter, don't disable. The search above is your starting point; refine it to the GUID once you confirm your index layout.

3 Timeline reads — four searches that tell the whole story

Instead of chasing single events, read the attack as a timeline. These four searches, run in order with a time window, reproduce the intrusion the way a security operations centre would.

3.1 The entrance

```
agent.name:<SERVER> AND rule.level:>=8 AND !(data.win.system.eventID:(4662))
```

"Everything notable on the DC that is not routine DS access." First look of every shift.

3.2 The brute that opened the door

```
agent.name:<SERVER> AND data.win.system.eventID:4625
# then flip '4625' to '4624' in the same window to find the success that followed
```

3.3 The hands-on-keyboard part

```
agent.name:<SERVER> AND data.win.system.eventID:(4688 OR 4104)
```

3.4 The payoff

```
agent.name:<SERVER> AND data.win.system.eventID:4662
```

If you see 4625 → 4624 → 4688/4104 → 4662 in sequence from `10.0.0.30`, you are looking at a replay of Volume 4. That ordering is the fingerprint to hunt for.

4 Writing local rules for the gaps

Stock Wazuh is good, but your lab has two classic gaps. Volume 1 showed the `local_rules.xml` mechanics; here are the two rules worth having.

4.1 The 4662 problem — detect DCSync even without full GUID tuning

If replication-file auditing is too fiddly, catch the *shape*: a single source object access read storm on the DS from a non-DC host. A frequency rule is the honest fix:

```
<rule id="100120" level="12" frequency="6" timeframe="120">
  <if_sid>60122</if_sid>
  <same_source_ip />
  <description>Rapid LDAP object access from one source - possible DCSync enumeration</description>
  <mitre><id>T1003.006</id></mitre>
</rule>
```

- `frequency="6" timeframe="120"` — six matching events within two minutes.
- `same_source_ip` — the six must come from one host.
- `if_sid 60122` — an existing logon/DS rule to chain onto; adjust to a rule your install actually fires (search `data.win.system.eventID:4662` in Dev Tools → Rules).

4.2 Spray / brute-force frequency rule (catch it before lockout)

```
<rule id="100121" level="10" frequency="8" timeframe="180">
  <if_sid>60122</if_sid>
  <same_source_ip />
  <description>Eight or more Windows logon failures from one source in 3 minutes</description>
  <mitre><id>T1110.003</id></mitre>
</rule>
```

4.3 Deploy and verify

```
sudo /var/ossec/bin/wazuh-logtest -t
sudo systemctl restart wazuh-manager
sudo /var/ossec/bin/agent_control -R 002      # agent 002 = WinServer; refresh its config
```

Then re-run the spray from Volume 4 and watch `100121` fire around guess eight. A rule that never fires in Wazuh is a rule you never tested.

5 Tuning: from "everything" to "everything interesting"

- **Start at level 8.** Low-severity noise (cron, sudo, routine logins) will drown you. Triage at `rule.level:>=8`, drill down only when investigating.
- **Filter to the agent that matters.** A lab with Kali + DC + client is three very different noise profiles. Separate them with `agent.name:<HOST>`.
- **Baseline before you attack.** Spend one hour just watching; what fires normally becomes boring, and only the deltas deserve your attention.
- **Suppress, don't disable.** When a rule is useless, lower it in `local_rules.xml` rather than deleting the group, so you can raise it later.
- **Alert fatigue is the enemy.** If everything is critical, nothing is. Ten tuned medium rules beat 200 shrieking ones.

6 Keeping the pipeline alive

A SIEM you do not maintain is a screenshot, not a security tool. Two things keep this lab honest: know the arrow, and know the failure that broke it once before.

6.1 The arrow

Agent → manager → alerts.json → filebeat → indexer → dashboard

When something does not appear in the UI, debug along this arrow in this order.

6.2 The one that bit you (and now lives in your playbook)

filebeat crashed at boot with a seccomp error — fixed 10 Oct 2026

Symptom: `wazuh-manager/indexer/dashboard` all `active`, filebeat `failed`, dashboard showing no new alerts. Logs screamed `runtime/cgo: pthread_create failed: Operation not permitted` — filebeat's built-in seccomp filter does not allow the `clone3` syscall that glibc uses on a new kernel (Ubuntu 26.04 / kernel 7.0), so the very first extra thread kills the process.

Fix applied: appended to `/etc/filebeat/filebeat.yml`:

```
seccomp:
  enabled: false
```

Then `sudo systemctl reset-failed filebeat` and `sudo systemctl start filebeat`. Credentials resolve from the filebeat keystore (`/var/lib/filebeat/filebeat.keystore`), not the config.

6.3 Routine health check (one line)

```
systemctl is-active wazuh-manager wazuh-indexer wazuh-dashboard filebeat
sudo curl -sk -u 'admin:<your-admin-pass>' 'https://127.0.0.1:9200/_cluster/health?pretty'
```

Four `active` + cluster `green` = the pipeline is fine and any quietness is a detection problem, not an infrastructure problem.

7 Cheat sheet of searches

Question	Search
Everything high on the DC	`agent.name:<SERVER> AND rule.level:>=8`
Failed logons	`agent.name:<SERVER> AND data.win.system.eventID:4625`
Successful logon	`agent.name:<SERVER> AND data.win.system.eventID:4624`
Process creation	`agent.name:<SERVER> AND data.win.system.eventID:4688`
Kerberos activity	`agent.name:<SERVER> AND data.win.system.eventID:(4768 OR 4769 OR 4771)`
Lockouts	`agent.name:<SERVER> AND data.win.system.eventID:4740`
Account changes (impact)	`agent.name:<SERVER> AND data.win.system.eventID:(4720 OR 4732 OR 4726)`
DCSync attempt	`agent.name:<SERVER> AND data.win.system.eventID:4662`
PowerShell	`agent.name:<SERVER> AND data.win.system.eventID:4104`
From the attacker IP	`data.win.eventdata.ipAddress:10.0.0.30`

Appendix A Coverage table for your report

Fill the last two columns over your next campaign. A detection that fired, with an event ID you can prove, is worth a hundred slides.

#	Technique	Event ID(s)	Search in Wazuh?	Fired in run?	Notes
1	Brute force / spray	`4625, 4740`			
2	AS-REP roast	`4768`			
3	Kerberoast	`4769`			
4	Shell (WinRM/SMB)	`4624`			
5	Command execution	`4688`			
6	DCSync	`4662`			
7	Account creation	`4720, 4732`			

The loop is complete.

Attack it (Volume 4), detect it (this volume), fix it (Volume 2/3's remediation), and re-run. A repo with all five documents is not just study notes — it is a finished portfolio artifact. The jobs you are aiming for spend their days exactly here.